



Shell Programming

Amitabha Sanyal

April 29, 2025



Contents

- Introduction, shebang and read
- Variables
- If statements and conditionals
- Case statements and glob patterns
- While loops
- For loops
- Select statement
- Shell functions
- Shell environment and visibility
- References



Shell programming and bash scripts

- **Bash** is the interpreter of a programming language called the POSIX Shell Scripting Language.
- Enables a user to combine the commands seen earlier to form programs
- The language has:
 - single commands
 - variables and their values, expressions
 - input - prompting user and command line arguments, and output
 - Decision -- **if-then-else**, **case**
 - Repetition -- **do-while**, **repeat-until**, **for**, **select**
 - Functions
- Workflow: Develop shell programs in a file called a bash script (typically with **.sh** extension), make the file executable and run.



Shell environment and shebang

- Shebang line for bash shell script:
 - **#!/bin/bash**
 - Should be the first line in the file.
- Specifies the environment in which the program should be run.
 - For portability
- Make the shell script executable with **chmod**
 - execute as: **./filename.sh**



Input using read

```
#!/bin/bash
```

```
read -p "May I ask your name: " name # -p is for prompt
echo "Hello $name"                  # $name is value of name.
exit 0                             # Normal exit
```

```
read -p "May I ask your name: "
echo "Hello $REPLY"                  # Try using ' instead of "
exit 0
```

- In general: **read -p "prompt" varname...**
- A first look at variables:
 - **name** is a user defined variable
 - **REPLY** is a bash variable
 - **\$** means evaluate



Variables

- Environment variables. Examine using **env**

```
as@home:~$ env
LANG=en_IN
DISPLAY=:0
GNOME_SHELL_SESSION_MODE=ubuntu
COLORTERM=truecolor
USERNAME=as
DESKTOP_SESSION=ubuntu
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
:/usr/games:/usr/local/games:/snap/bin
OLDPWD=/home/as/Dropbox/cs251/bash
...and many more
```

[Here](#) is a discussion on environment variables.



\$ variables

- \$ variables (special shell variables)

Parameter	Meaning
\$0	Name of the current shell script
\$1-\$9	Positional parameters 1 through 9
\$#	The number of positional parameters
\$*	All positional parameters, "\$*" is one string
\$@	All positional parameters, "\$@" is a set of separate strings
\$?	Return status of most recently executed command
\$\$	Process id of current process

- User variables (like any other language) COUNT, i, j, k
 - assign as i=...
 - evaluate as \$i



\$ variables - example.

```
#!/bin/bash
```

```
echo $0 $1 $2 $3 $4      # Variables set at call point
```

```
echo $# $* $@ $$
```

```
set cat dog foo bar      # $1-$4 set to these values
```

```
echo $*
```

```
echo $PWD
```




Conditional expressions

- The most general form:

if **conditional**; **then**

statements

elif **conditional**; **then**

statements

else

statements

fi

- No surprises except for **conditional** expression



Conditional expressions

- Two ways of writing the same thing:
 - test expression
 - [expression] -- Notice the space around expression
- We shall use the [expression] form

- The **if** statement can also use any command as a conditional. **True** if command succeeds.
- Try:

```
#!/bin/bash

if ls -al; then
    echo 0
else
    echo 1;
fi
```



Conditional expressions - testing

Testing strings:

- | | |
|---------------------------|---|
| > [\$USER = root] | - Is the user root? |
| > [\$USER != root] | - Is the user not the root? |
| > [-n \$SSH_TTY] | - Is SSH_TTY defined (non-zero length)? |
| > [-z \$SSH_TTY] | - Is SSH_TTY defined (zero length)? |
| > [string1 \< string2] | - Is string1 less than string2 |
| > [[string1 < string2]] | |

Testing integers:

- | | |
|-----------------|---|
| > [\$# -gt 0] | - Can also have -ge, -lt, -le, -eq, -ne |
|-----------------|---|

Testing files:

- | | |
|-------------------------|-----------------------------|
| > [-h /usr/src/linux] | - File is a symbolic link |
| > [-d /usr/src/linux] | - File is a directory |
| > [-x /usr/src/linux] | - File is executable |
| > [file1 -nt file2] | - file1 is newer than file2 |

A complete list of test expressions appear [here](#).

Proprietary content. Copyright © 2012 Pearson Education, Inc. All rights reserved. Unauthorized use or distribution prohibited. Sharing or publishing the contents in part or full is liable for legal action.



Example:

```
#!/bin/bash
```

```
read -p "Passed in all core courses in last two semesters? (Y/N) :  
" Passed  
read -p "Enter rounded CPI: " CPI  
read -p "Enter total course credits: " Credits  
if [ $Passed = Y ] && [ "$CPI" -ge 8 ]; then  
    echo "You can register for 54 credits"  
elif [ $Credits -gt 48 ]; then  
    echo "Please drop some courses."  
else  
    echo "Go ahead and register"  
fi
```

- Notice the use of the **&&** (and) operator. In addition, the operators **||** (or) and **!** (not) are also available.
- **&&** and **||** should be outside **[...]** test can handle **!** but not **&&** and **||**



Conditional expressions

```
#!/bin/bash
```

```
    if [ $# -lt 1 ]; then  
        echo "Usage: filetest.sh filename"  
        exit 1
```

```
fi
```

```
if [[ ! -f "$1" || ! -r "$1" || ! -w "$1" ]] # Notice: [...] ]]  
then
```

```
    echo "File $1 is not accessible"  
    exit 1
```

```
fi
```



case statement

```
#!/bin/bash
```

```
read -n1 -p "Hit a key: " key
```

```
echo
```

```
case $key in
```

```
    [a-z]) echo "$key is a lower case" ;;
```

```
    [A-Z]) echo "$key is an upper case" ;;
```

```
    [0-9]) echo "$key is a digit" ;;
```

```
    *)      echo "$key is some thing else" ;;
```

```
esac
```



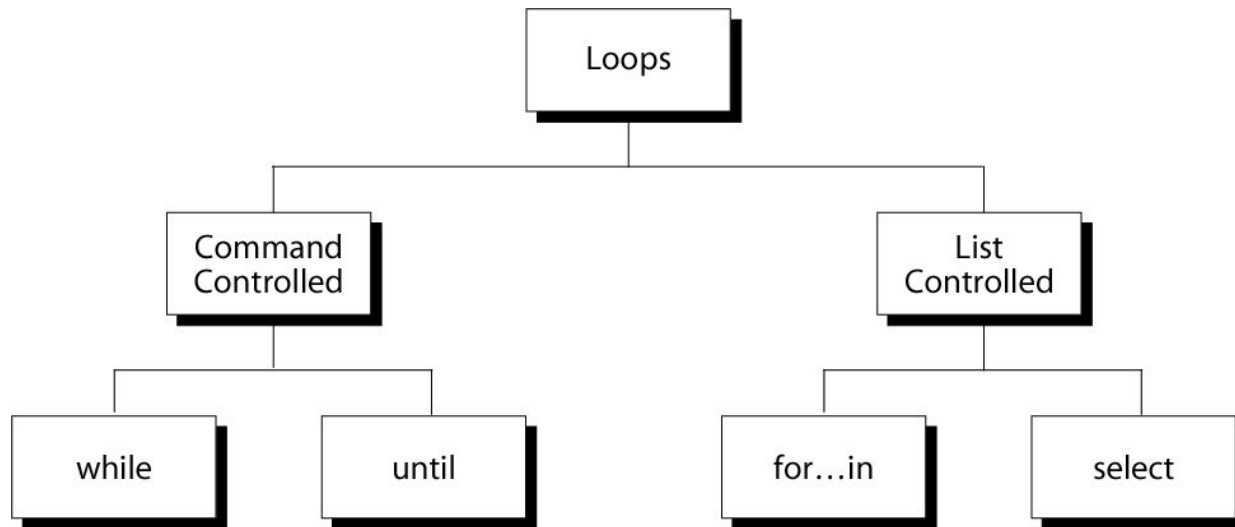
patterns (globs and extglob)

<code>'What now?'</code> <code>*</code> <code>?</code> <code>[[:alpha:]]</code> <code>[yY][eE][sS]</code>	Matches the string <code>'What now?'</code> Matches a string with zero or more characters. Matches single character. <code>a.c??</code> matches <code>a.cvs</code> and <code>a.cpp</code> . Strings consisting of a single alphabet. Matches yes , YES , YeS , etc.
<code>(pattern1 pattern2)</code>	strings that match <code>pattern1</code> or <code>pattern2</code> . Becomes a pattern when prefixed by a <code>extglob</code> operator
<code>@(pattern)</code> <code>?(pattern) ,</code> <code>*(pattern) ,</code> <code>+(pattern) ,</code> <code>!(pattern)</code>	strings that match <code>pattern</code> . strings that match 0 or 1 occurrences of <code>pattern</code> strings that match 0 or more occurrences of <code>pattern</code> strings that match 1 or more occurrences of <code>pattern</code> strings that do not match <code>pattern</code>

- `extglob` enabled with `shopt -s extglob`, disabled with `shopt -u extglob`



LOOPS





Command controlled loops: while

```
while command
do
    commands
done
```

```
while read lineoftext
do
    if [ -f "$lineoftext" ]
    then
        echo "$lineoftext is a file"
    else
        echo "$lineoftext is not a file"
    fi
done
```



while loops

```
read -p "Give me a number: " n
i=1
fac=1

while (( i <= n ))      #no $
do
    fac=$(( fac * i )) #notice the $
    ((i++))             #no $
done
echo $fac
```

- `$((...))` is the evaluation of an arithmetic expression to a value, which is substituted in place.
- `((...))` is an arithmetic expression evaluated either for its side-effects or its exit status.
 - The exit status is the value of the expression changed to a 0 if the value is non-zero and 0 otherwise.



while loops

```
while IFS=: read userName passWord userID groupID genInfo  
homeDir userShell  
do  
    echo "$userName -> $homeDir"  
done < /etc/passwd
```

- A read with field bindings based on a separator.



for loop

```
for var in argument-list
do
    commands
done
```

```
TEMPIFS=$IFS
IFS=$'\n'
for args in $(ls -al)
do
    echo "$args"
done
IFS=$TEMPIFS
```

```
for args in *
do
    ls -al "$args"
done
```

```
for index in 1 2 3 4 5 6
do
    if [ $index -le 3 ]
    then
        echo "continue"; continue
    else
        echo "break"; break
    fi
done
```



select loop

```
#!/bin/bash

echo "script to make files private"
echo "Select file to protect:"

select FILENAME in *
do
    echo "You picked $FILENAME ($REPLY)"
    chmod go-rwx "$FILENAME" #retains user permissions
    echo "it is now private"
done
```

A discussion on loops in bash appears in [here](#).



Shell functions

```
show_system ()  
{  
    echo "The uptime is:" `uptime`  
    echo  
    echo "System details:" $(uname -a)  
    echo  
    echo "User list:" `who`  
}
```

```
show_system
```

```
check_file() {  
    if [ -f "$1" ]; then  
        return 0 # Success  
    else  
        return 1 # Failure  
    fi  
}
```

```
check_file "/etc/passwd"  
status=$? # Captures the  
return value  
echo "The exit: $status"
```



Text processing and others

```
var1='A'  
var2='B'
```

```
my_function () {  
    local var1='C'  
    var2='D'  
    echo "Inside function: var1: $var1, var2: $var2"  
}
```

```
echo "Before executing function: var1: $var1, var2: $var2"  
my_function  
echo "After executing function: var1: $var1, var2: $var2"
```



Importing names through source

file1.sh

```
my_function ()  
{  
    echo "some result"  
    return 55  
}
```

terminal

```
> source file.sh  
> my_function  
"some result"  
> echo $?  
55
```

- Source imports all the variables and functions of file.sh into the terminal environment. Executes in the terminal shell.



Creating a new shell: child shell

`file1.sh`

```
my_function ()
{
    echo "some result"
    return 55
}

export -f my_function
./file2.sh
```

`file2.sh`

```
env | grep "my_function"
my_function
```

- `./file.sh` or `bash file.sh` or executing a non-builtin (e.g `grep`) creates a child shell.
- These processes are created via `fork()` followed by `exec()`.
 - The `exec` call wipes the inherited memory and starts fresh.
- Child shells **can** only see environment variables of parents `PATH`, `HOME`
- They **can** see the parent's local variables or functions only if they are exported using `export` or `export -f`



Creating a new shell: subshell

A subshell is created from a parent shell:

- Using parentheses `(...)` around commands.
- Using command substitution: `$(...)` or backticks ``...``
- Using commands in pipelines, only if the command is a built-in (part of bash)
- These processes are created via a `fork()` system call only (no `exec`).
 - They inherit a complete copy of everything—including local variables and functions—but cannot pass changes back to the parent.

```
x=5
( x=$((x+1)); echo "Inside subshell: x=$x" )
echo "Back in parent: x=$x"
```

- If the command in `(...)`, `$(...)`, or ``...`` is not a built-in, then a child-shell is forked.



Bash shell environment and visibility

What is inherited by a subshell from the parent shell?

- All *exported environment variables* `PATH`, `HOME`
 - Shell variables `$1`, `$PPID`, `$PS1`
 - User Variables `x`, `my_function`
 - Current working directory
 - File descriptors (`stdin`, `stdout`, etc.)
-
- What is inherited by the parent shell from a subshell?
 - *Nothing*. The parent shell does **not** see variables or state changes made in the subshell.
 - The child communicates its result to the parent through a pipe.



References

1. [GNU Bash manual - GNU Project - Free Software Foundation](#) The official documentation from the GNU Project.
2. [BashGuide - Greg's Wiki](#) Commonly known in the community as "The Bash FAQ,"
3. [ShellCheck](#). You paste your code in, and it tells you exactly where your process logic might fail.
4. [Advanced Bash-Scripting Guide](#) It covers the "heavy" stuff: Process Substitution, Restricted Shells, and complex I/O Redirection
5. [The Architecture of Open Source Applications \(Volume 1\)The Bourne-Again Shell](#) Discusses how Bash is built internally
6. [Bash Guide for Beginners](#) – The Linux Documentation Project.